



VSCode and GitHub for Practical Collaboration

MultiVerse Data Lab - 30th June 2026

Presented by: Laura Martin
[linked.in/laura-joanne-martin](https://www.linkedin.com/in/laura-joanne-martin)

VS Code and GitHub

VS Code

A free, Microsoft-developed and maintained application used to write, edit, organise, and manage code, documents, and technical projects.



GitHub

An online platform that stores project files and allows teams to track changes, collaborate, and manage version-controlled projects.



Git vs GitHub

Git

Git is an **open-source** version control tool created in 2005 by developers working on the Linux operating system. You do not need GitHub to use git, but you cannot use GitHub without using git.



GitHub

An online platform that stores project files and allows teams to track changes, collaborate, and manage version-controlled projects. **It uses Git as an open-source tool.**



The 'why' of using GitHub and VSCode together

- Why collaboration matters in tech roles
- Why email and/or file versions fail
- What problems Git/GitHub solve
- Where VSCode fits into the workflow
- CI/CD as a concept and way of working.

Why Collaboration Matters in Tech Roles

- **A typical project may involve:**
 - Subject matter experts
 - Business/Data analysts
 - Developers
 - Cloud engineers
 - Cyber security teams
 - Project managers
 - End users
- **Why collaboration matters**
 - Share knowledge and expertise
 - Reduce duplicated work
 - Improve quality through review
 - Maintain continuity when people leave or change roles
 - Deliver larger and more complex projects

Why Email and File Versions Fail

- **What Usually Happens?**

-  ProjectReport.docx
-  ProjectReport_v2.docx
-  ProjectReport_v3_Final.docx
-  ProjectReport_v3_Final_ReallyFinal.docx
-  ProjectReport_v3_Final_ReallyFinal_USE_THIS_ONE.docx

- **Common Problems**

- Nobody knows which version is current
- Changes can be overwritten
- Difficult to see who changed what
- Difficult to roll back mistakes
- Time wasted comparing files
- Knowledge lost in personal files when people leave

What Problems Do Git and GitHub Solve?

- **Version Control for Teams**

- Git tracks changes to files over time.
- GitHub provides a platform for teams to share and collaborate using Git.



- **What Can Git Do?**

- Track every change made
- Record who made each change
- Restore previous versions
- Allow multiple people to work simultaneously
- Compare changes easily
- Maintain a complete project history regardless of time and complexity

Where Does VS Code Fit In?

- **Visual Studio Code (VS Code) is where many technical professionals do their day-to-day work. In VS Code, you can:**
 - Edit code and files
 - View project structure
 - Connect to Git and GitHub
 - Review changes
 - Create commits
 - Manage branches
 - Work with documentation
 - Run and test projects

CI/CD - A Modern Way of Working

- **CI/CD = Continuous Integration & Continuous Delivery**
- **CI/CD helps teams move changes from idea to testing to production (live, deployed versions) safely and consistently.**
- **Benefits**
 - Faster feedback
 - Reduced human error
 - Consistent deployments
 - Improved quality
 - More frequent releases
- Not just code - dashboards, reports, CAD models, and more increasingly use CI/CD-inspired approaches.

GitHub: Overview



The screenshot shows the GitHub interface for a repository named "Multiverse_VSCode_GitHub_Practical_Collaboration" owned by "lauramartinazure". The repository is public. The top navigation bar includes links for Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, and Settings. The repository header shows the name, a "Public" badge, and buttons for Pin, Watch (0), Fork (0), and Star (0). Below the header, the main content area displays the "main" branch with 1 branch and 0 tags. A search bar "Go to file" is present. The commit history shows an "Initial commit" by "lauramartinazure" with hash "5d96e4c" from "last week" and "1 Commit". A file named "README.md" is listed as part of the initial commit. The README content is displayed below, featuring the repository name and a description: "Repo for Multiverse session on using VSCode and GitHub for practical team collaboration on software projects." The right sidebar contains an "About" section with a description: "Repo for Multiverse session on using VSCode and GitHub for practical team collaboration on software projects." Below this are statistics: 0 stars, 0 watching, and 0 forks. A "Releases" section at the bottom indicates "No releases published" and provides a link to "Create a new release".

lauramartinazure / Multiverse_VSCode_GitHub_Practical_Collaboration

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Multiverse_VSCode_GitHub_Practical_Collaboration Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file

Add file Code

lauramartinazure Initial commit 5d96e4c · last week 1 Commit

README.md Initial commit last week

README

Multiverse_VSCode_GitHub_Practical_Collaboration

Repo for Multiverse session on using VSCode and GitHub for practical team collaboration on software projects.

About

Repo for Multiverse session on using VSCode and GitHub for practical team collaboration on software projects.

- Readme
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases

No releases published

[Create a new release](#)



 lauramartinazure / Multiverse_VSCode_GitHub_Practical_Collaboration

[Code](#) [Issues](#) [Pull requests](#) [Agents](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

 **Multiverse_VSCode_GitHub_Practical_Collaboration** Public [Pin](#) [Watch 0](#) [Fork 0](#) [Star 0](#)

 main

 1 Branch

 0 Tags

[Add file](#) [Code](#)

 **LauraMartinNQCC** Addition of a small demonstration script that reads a CSV 901c4af · 6 minutes ago 2 Commits

 README.md	Addition of a small demonstration script that reads a CSV	6 minutes ago
 demo_script_1.py	Addition of a small demonstration script that reads a CSV	6 minutes ago

 **README**

Multiverse_VSCode_GitHub_Practical_Collaboration

Repo for Multiverse session on using VSCode and GitHub for practical team collaboration on software projects.

Session Content:

The 'why' of using GitHub and VSCode together:

- Why collaboration matters in tech roles
- Why email and/or file versions fail

About

Repo for Multiverse session on using VSCode and GitHub for practical team collaboration on software projects.

 Readme

 Activity

 0 stars

 0 watching

 0 forks

Releases

No releases published

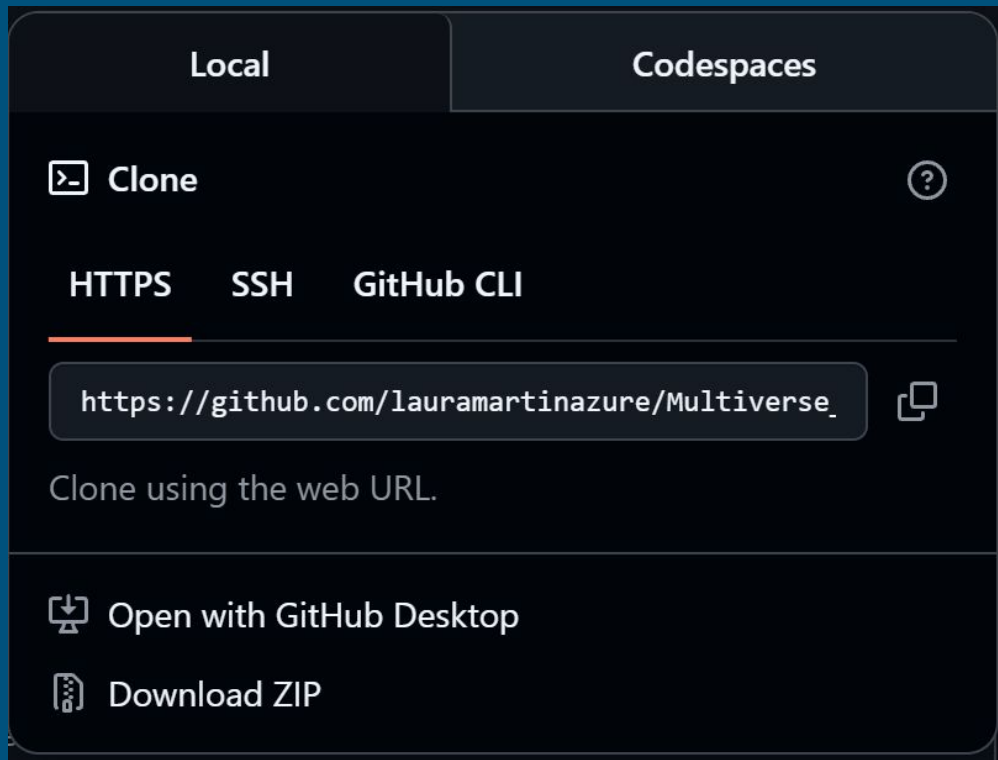
[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 2



Cloning

Cloning is the process of copying a repo down from GitHub to your local machine - and creating the git files that allow you to synchronise with the 'remote' repo.



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8390]
(c) Microsoft Corporation. All rights reserved.

C:\Users\[redacted] git clone https://github.com/lauramartinazure/Multiverse_VSCode_Git
Hub_Practical_Collaboration.git
```

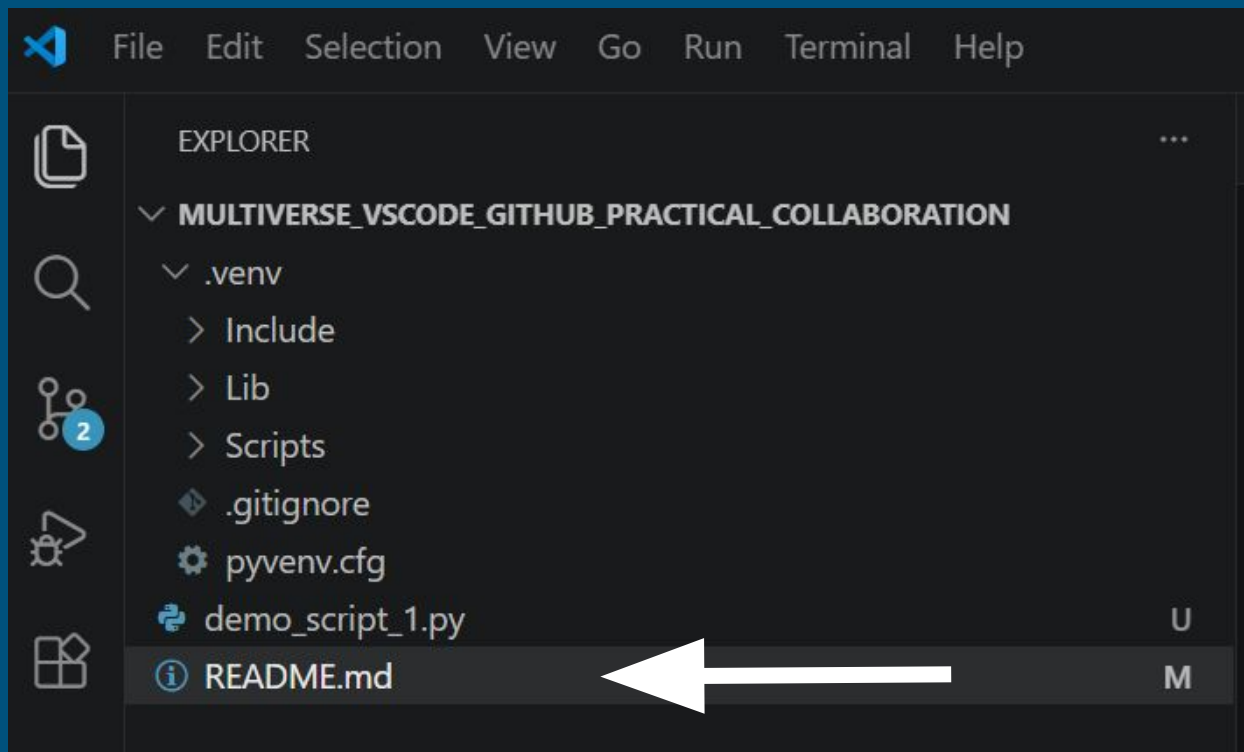
Cloning

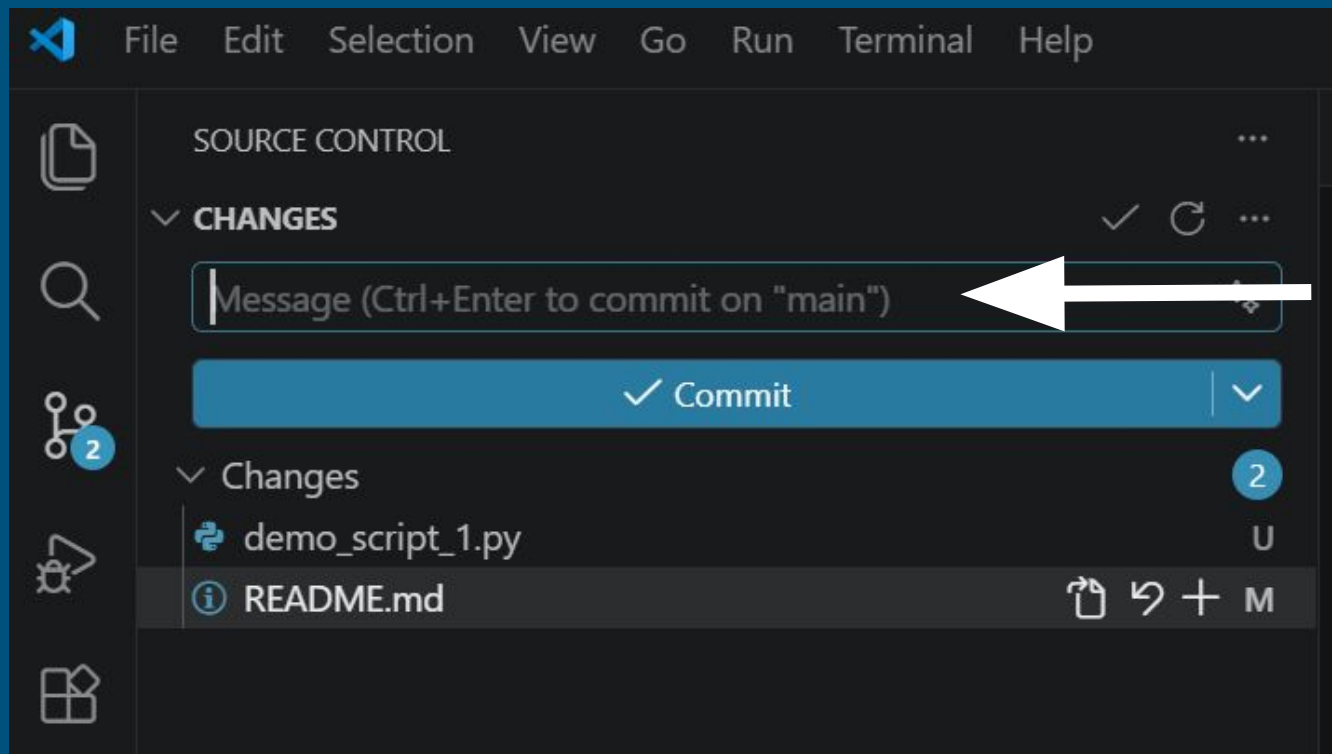
You can also clone using the command prompt locally.

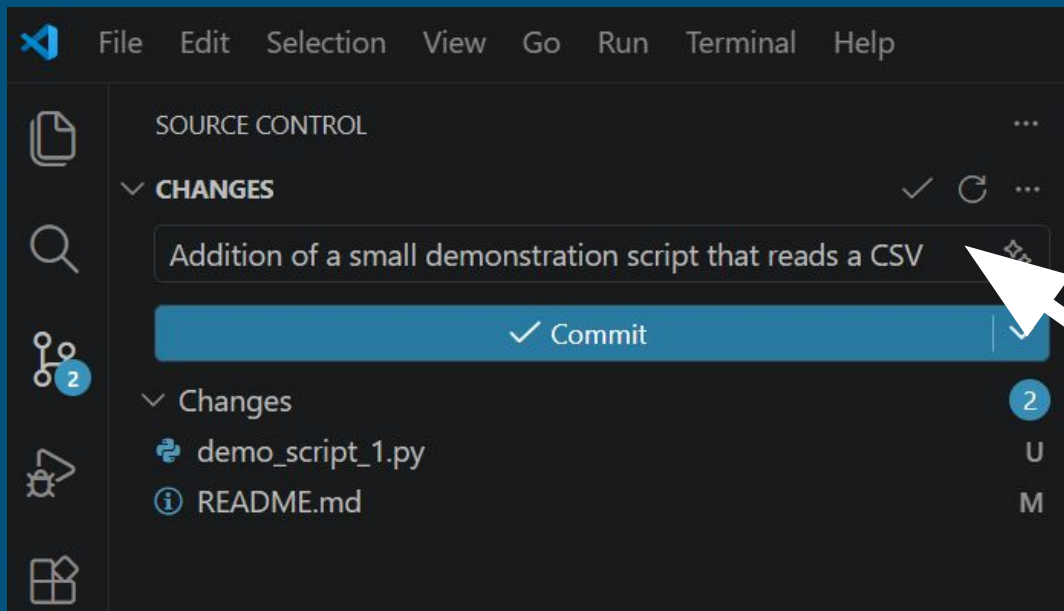


```
git clone <link to repo>
```

```
git clone https://github.com/org/repo.git
```







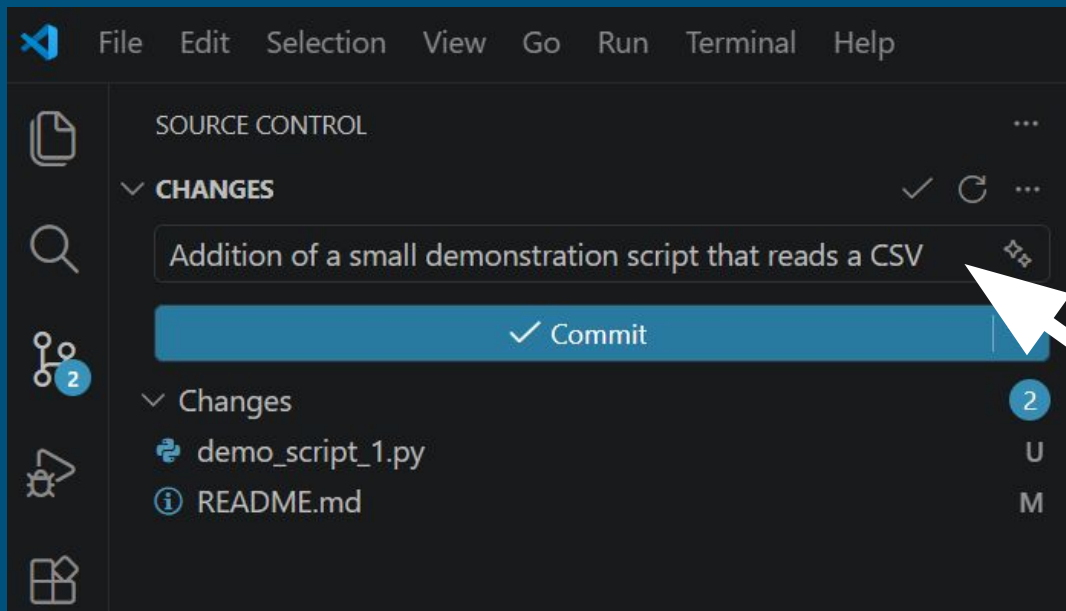
Commits

A way to create a specific 'waypoint' in your changes. Analogous to saving your changes (although you can save multiple times locally between each commit).



```
git clone <link to repo>
```

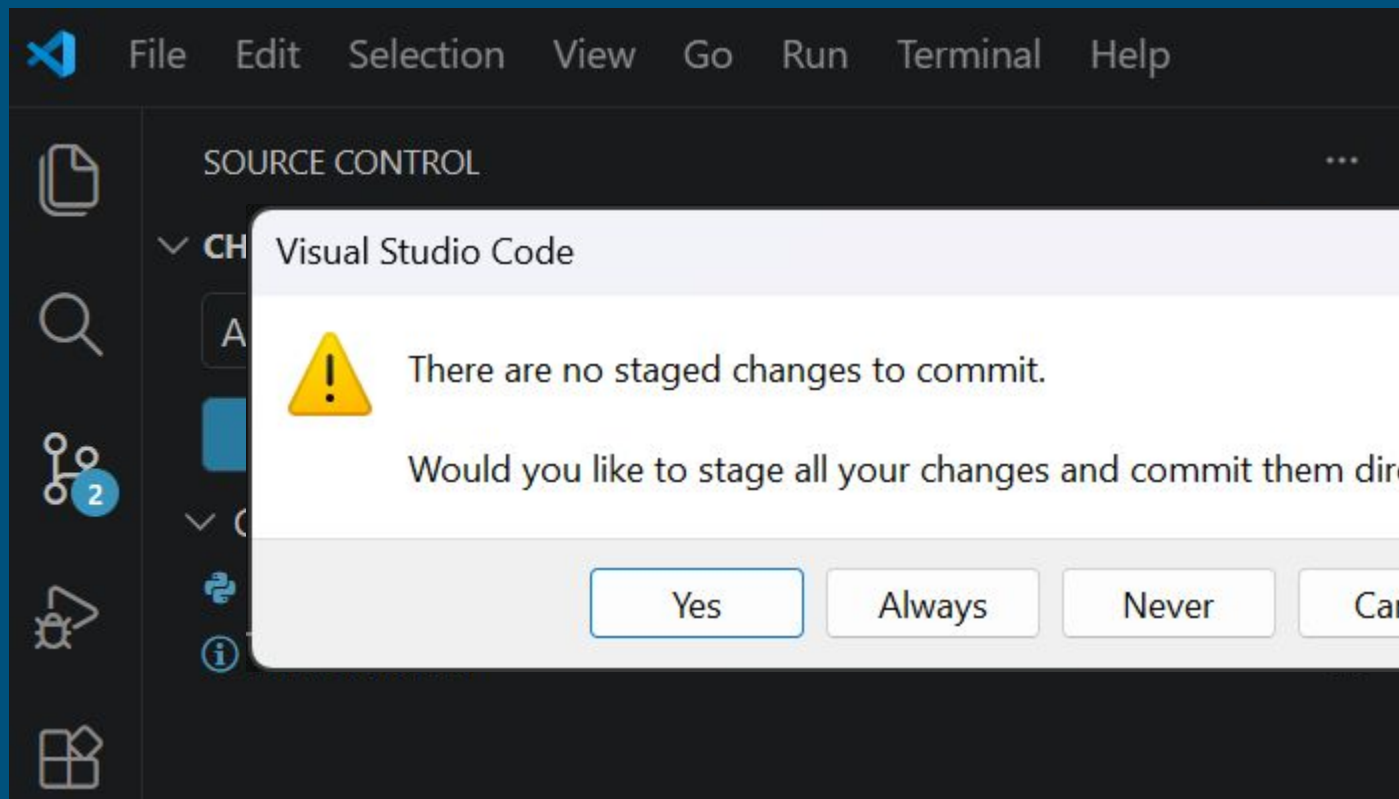
```
git clone https://github.com/org/repo.git
```

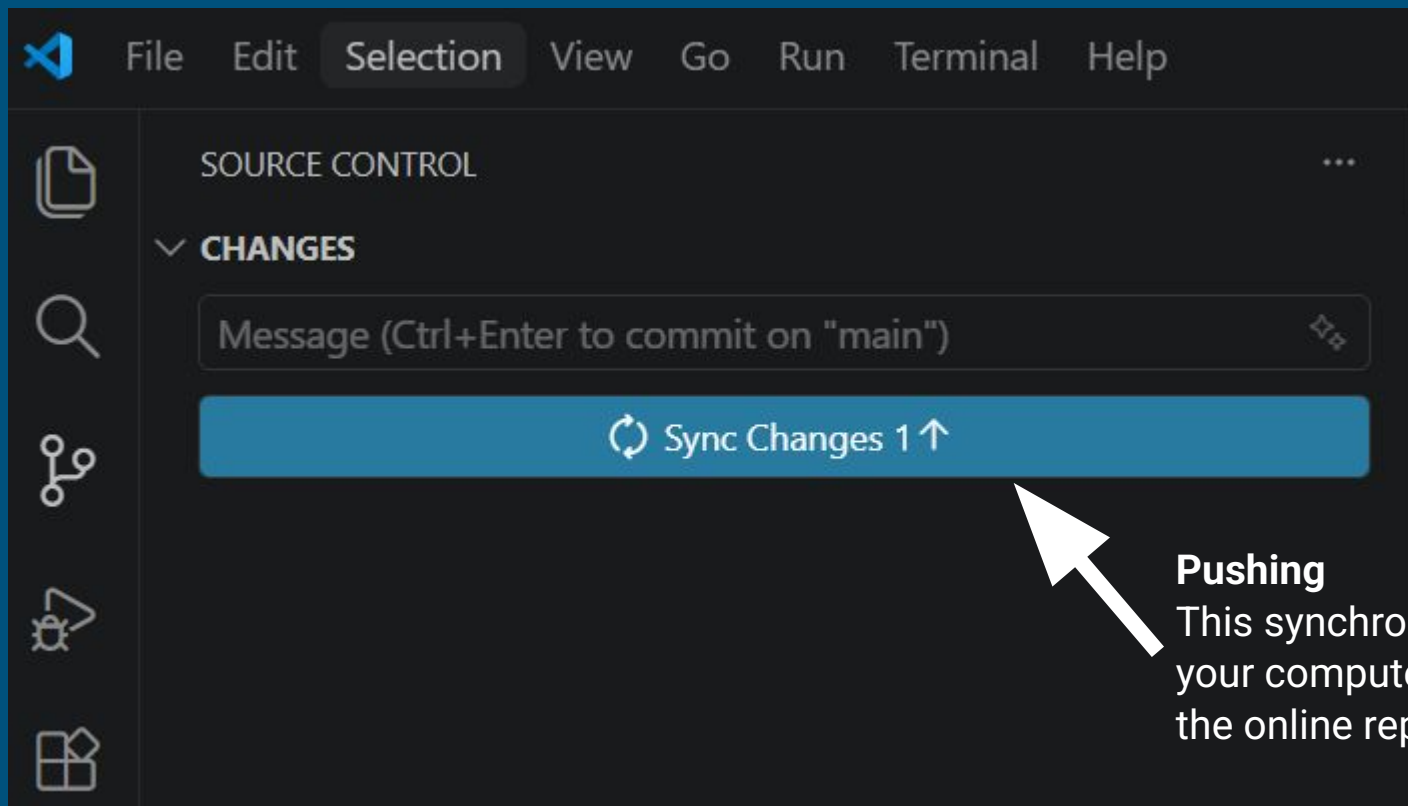


Staging

Staging is the process of selecting which changes you want to include in your next commit.

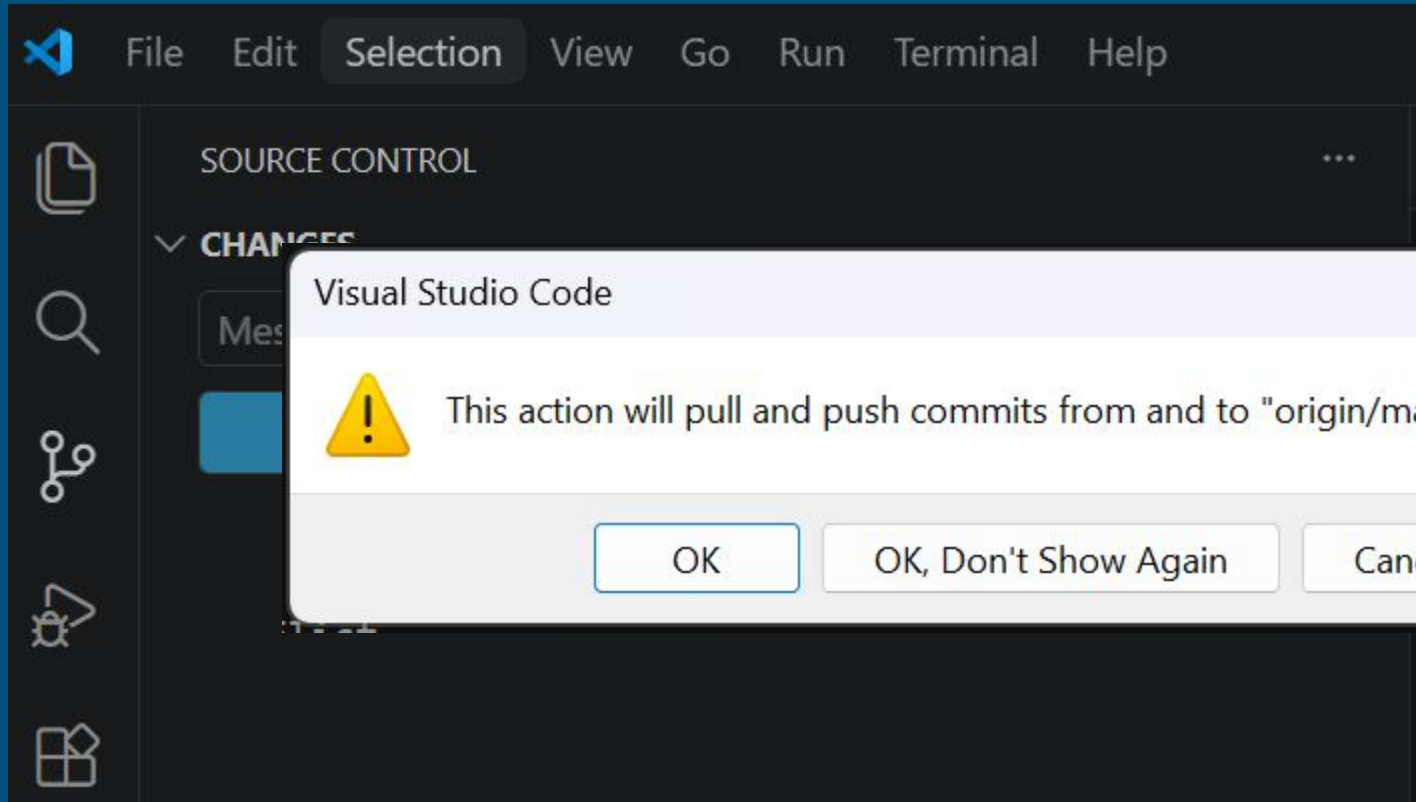
Imagine you've edited five files, but only three of them are ready to share with your team. Staging lets you choose those three files and leave the others for a later commit.

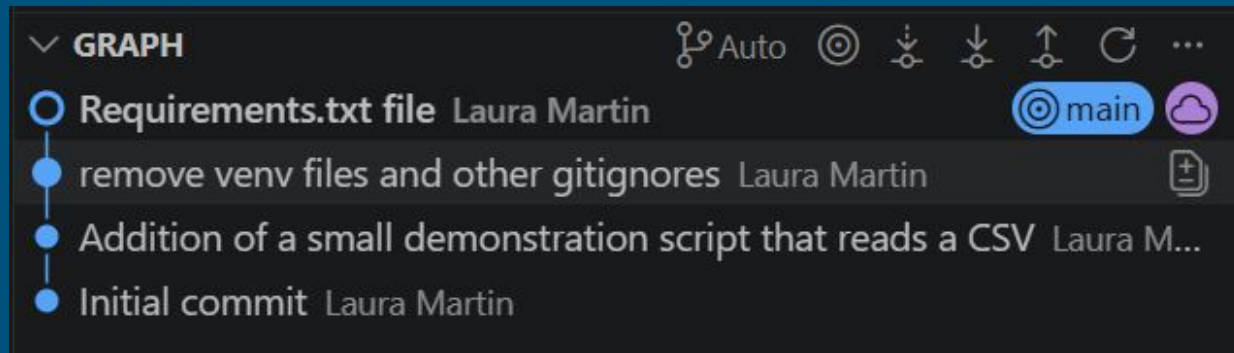
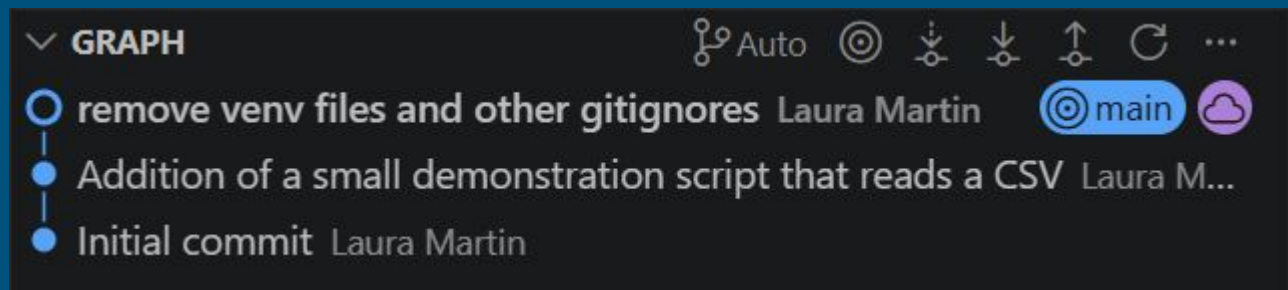
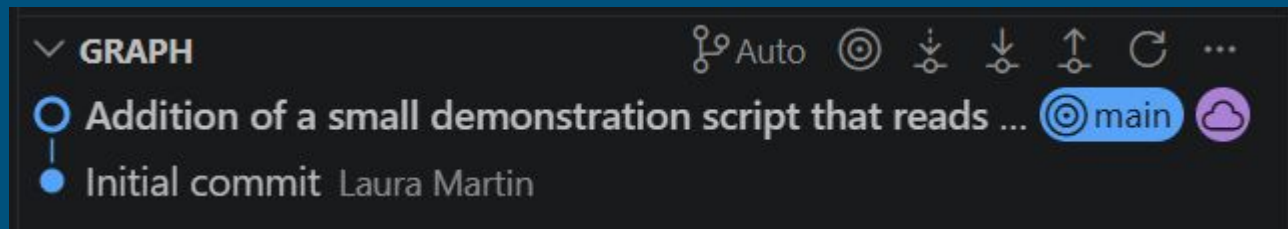




Pushing

This synchronises local (on your computer) changes with the online repository copy.

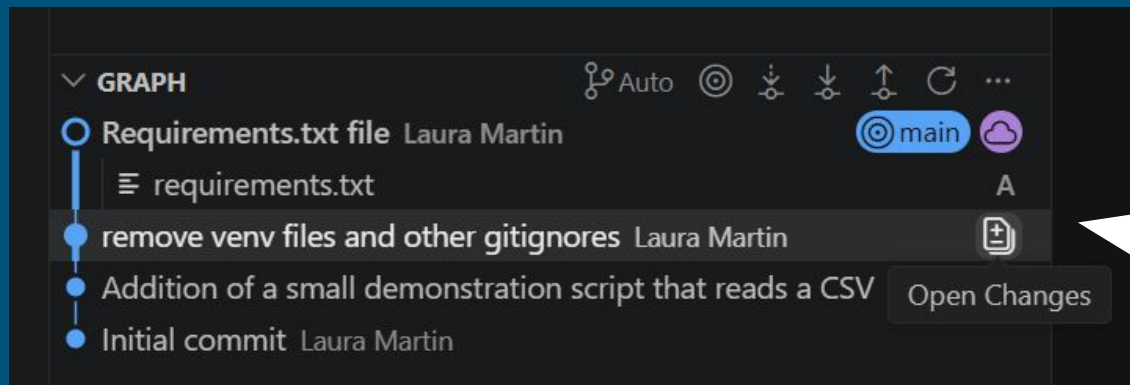




The Repo Graph

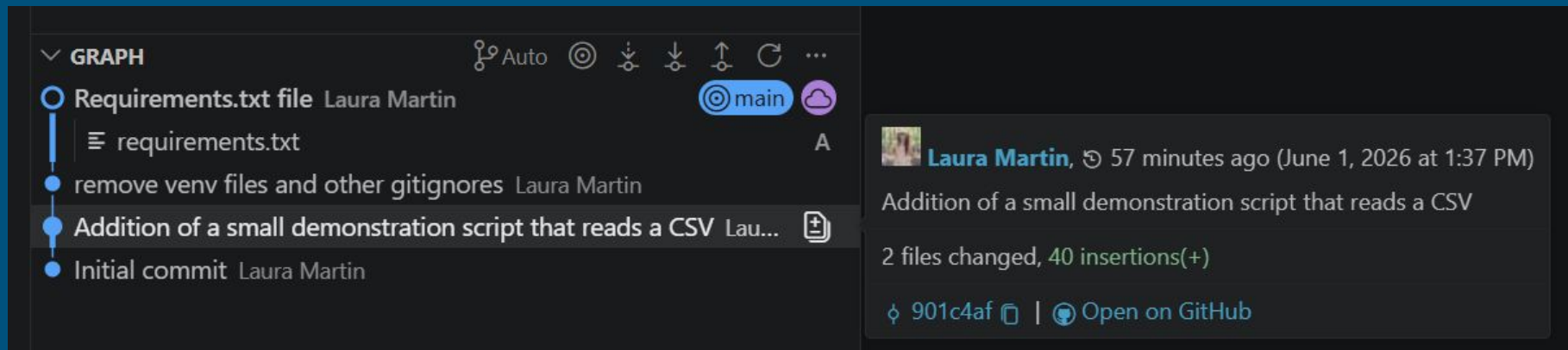
A visual representation of changes made over time. Each change has a dot, commit message, and author.

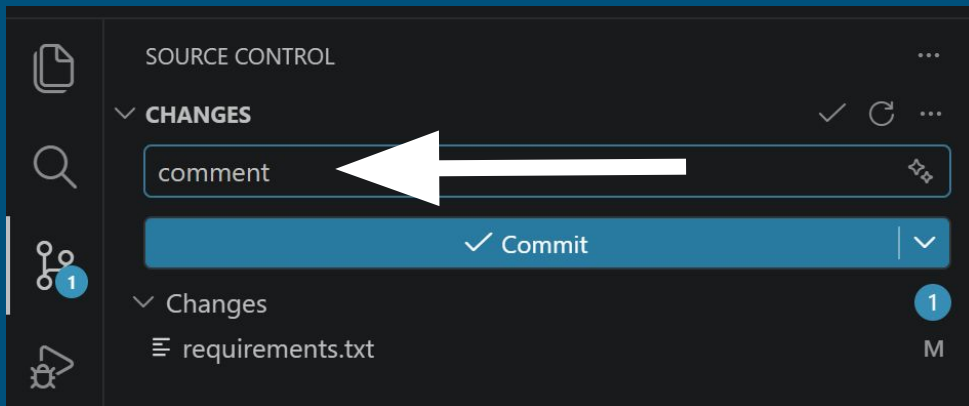
Sometimes referred to as a 'tree' instead.



Commits

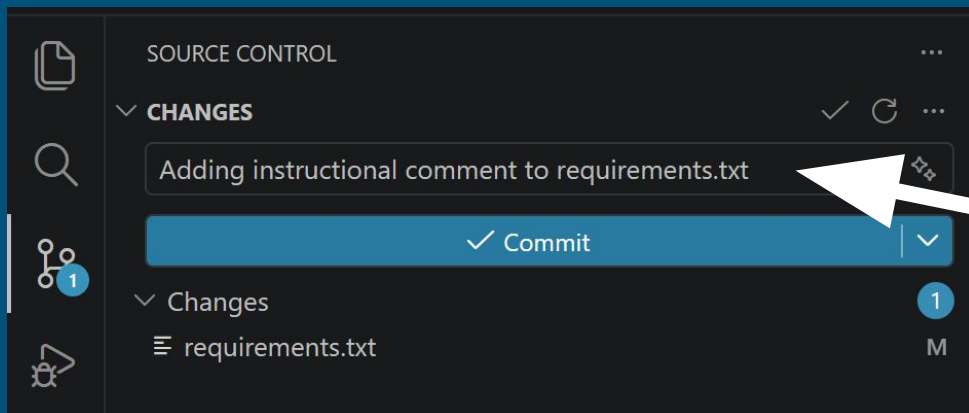
Each commit message is displayed in the repo history. This helps you track all the changes.





Bad Practice

Single words, 'test', keyboard mashes, lack of context or descriptions, only clear to original author.



Good Practice

Specific content, refers to individual file(s), describes change, clear to all readers.



Stage a specific file:

```
git add README.md
```

Stage multiple files:

```
git add README.md app.py
```

Stage all changes:

```
git add .
```



Commit staged changes:

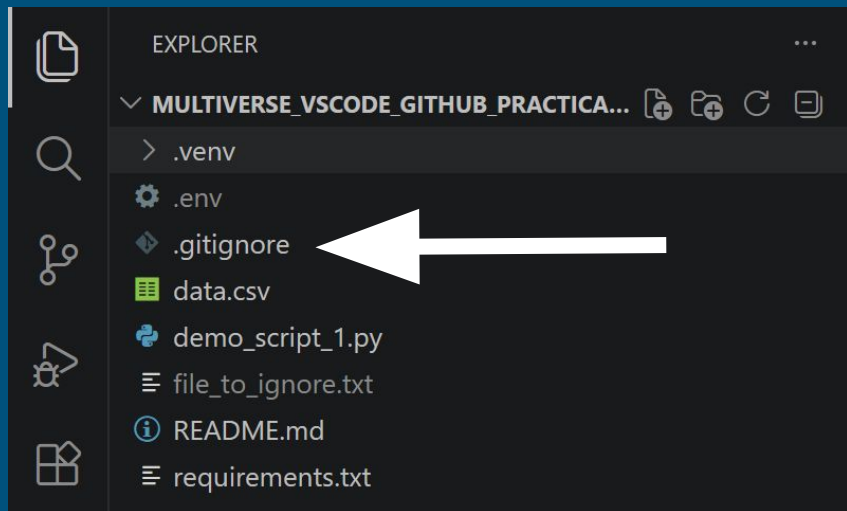
```
git commit -m "Add dashboard filter functionality"
```

Push current branch:

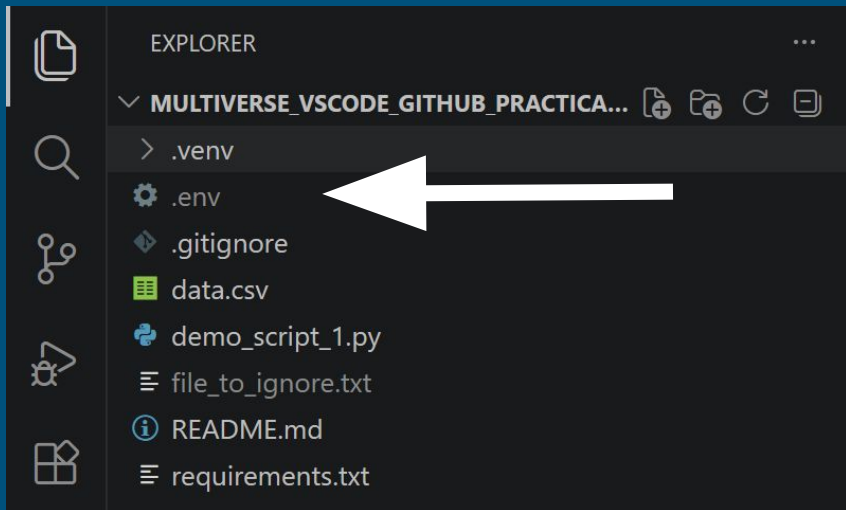
```
git push
```

First push of a new branch:

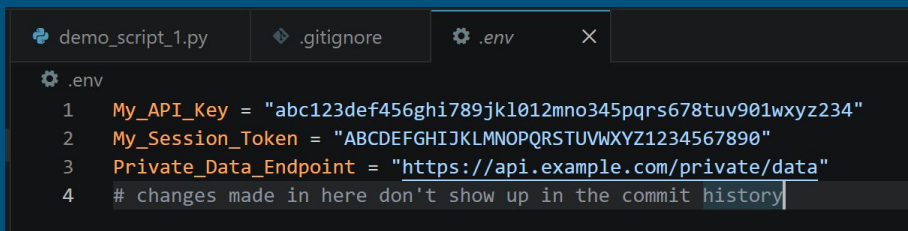
```
git push -u origin feature/new-dashboard
```



- **.gitignore tips**
- Make sure the files are added either *in the same commit* or *after* listing them in the gitignore.
- If the files pre-existed the .gitignore, run ``git rm –cached <filename>`` to remove the cached file.
- Make sure your .gitignore file is at the same folder level as the files you want it to ignore and/or paths to the file are correct.



- **.env file tips:**
- .env files should always be ignored!
- They are to contain sensitive information such as API keys, session tokens, personal IDs, private endpoints, secrets, keys, passwords, over-rides for ports ... anything you wouldn't want to be shared.





 **Multiverse_VSCode_GitHub_Practical_Collaboration** Public Pin Watch 0

main 1 Branch 0 Tags Add file Code

 **LauraMartinNQCC** Update .gitignore

64cedd6 · 12 minutes ago 15 Commits

 .venv	Moved gitignore into top folder level.	20 minutes ago
 .gitignore	Update .gitignore	12 minutes ago
 README.md	Addition of a small demonstration script that reads a CSV	20 hours ago
 data.csv	Adding instructional comment to requirements.txt	33 minutes ago
 demo_script_1.py	Instructional comments added to demo_script_1.py	13 minutes ago
 requirements.txt	Adding instructional comment to requirements.txt	33 minutes ago

**Ignored files
don't show up!**

Navigation bar: Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, More

Branches

[New branch](#)

Overview | Yours | Active | Stale | All

Search branches...

Default

Branch	Updated	Check status	Behind Ahead	Pull request
main	15 minutes ago		Default	

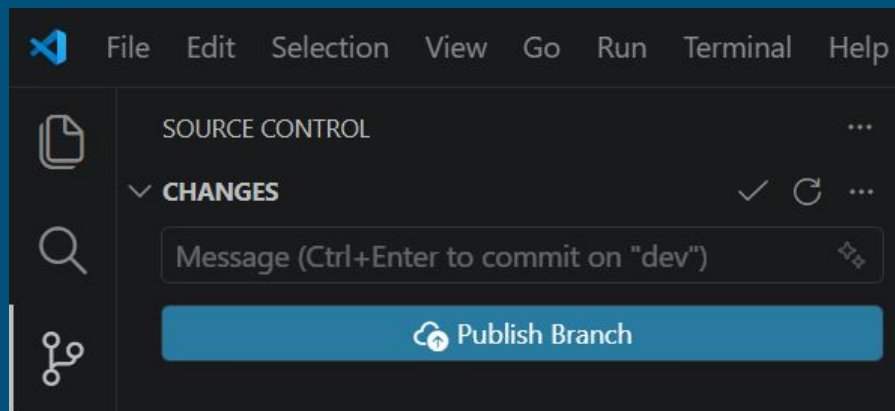
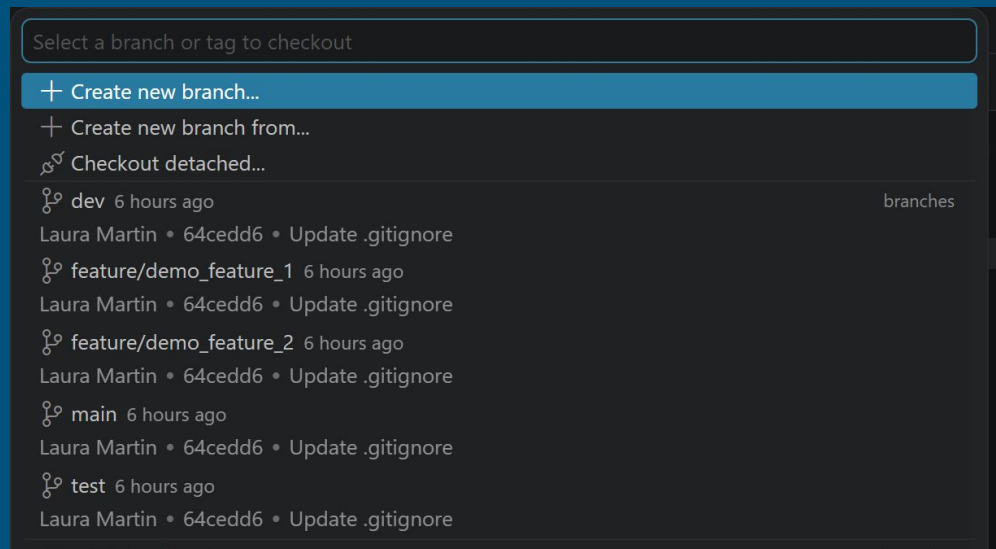


Creating a branch

Select a branch or tag to checkout

- + Create new branch...
- + Create new branch from...
- Checkout detached...
- main 17 minutes ago (branches)
- origin/main 17 minutes ago (remote branches)







Old version:

```
git checkout -b feature/new-dashboard
```

New version:

```
git switch -c feature/new-dashboard
```

View branches:

```
git branch
```



Multiverse_VSCode_GitHub_Practical_Collaboration

Type to search

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Branches

Overview

Yours

Active

Stale

All

Branch	Updated	Check status
main	6 hours ago	
test	now	
feature/demo_feature_2	now	
feature/demo_feature_1	now	
dev	now	

Create a branch

New branch name

feature/demo_feature_3_github

Source

main

Cancel

Create new branch



main



test



dev



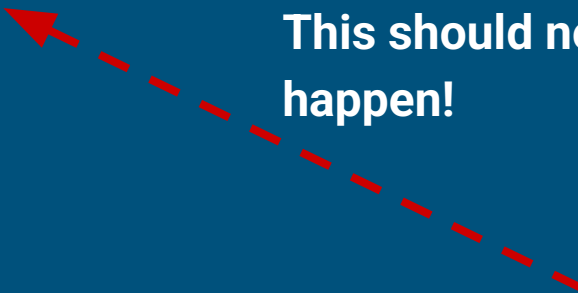
*Pull requests
and merges*

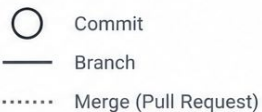
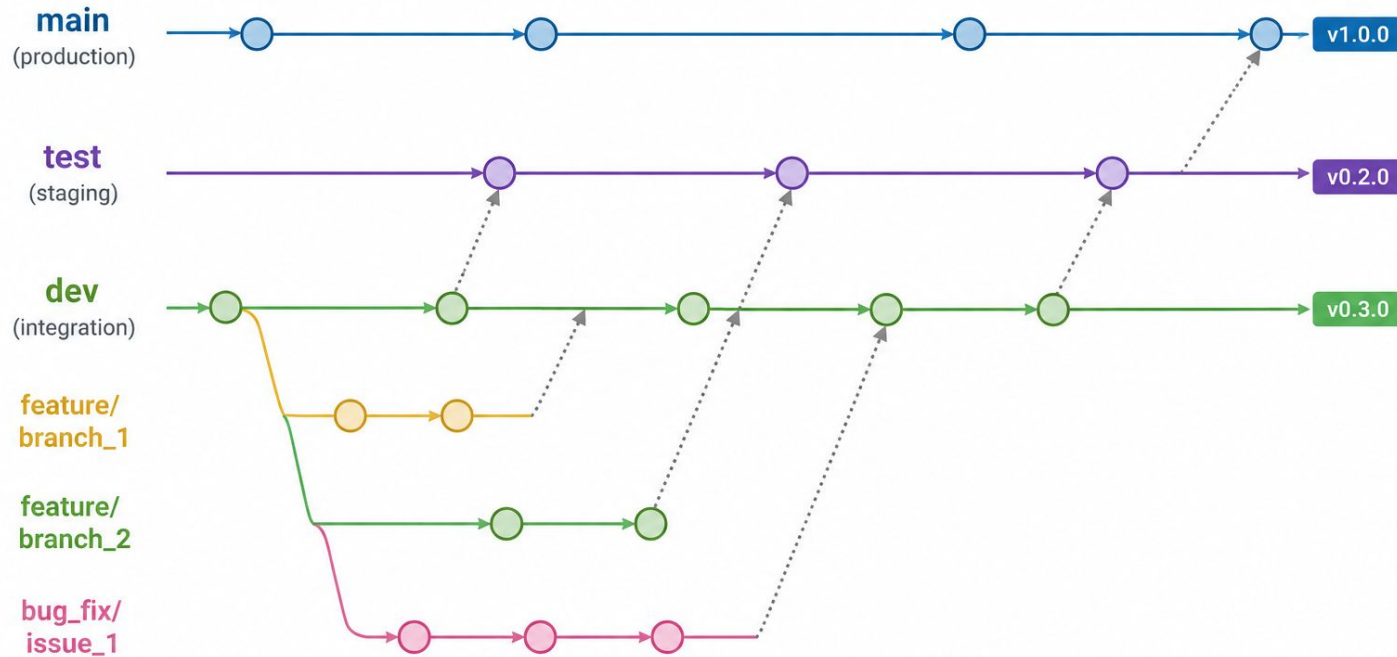
feature/branch_1

feature/branch_2

bug_fix/issue_1

This should not
happen!





1. Work happens in **feature branches** (or bugfix branches)
2. Changes are merged into **dev** via Pull Requests
3. Tested and verified changes are promoted from dev → test
4. When ready, **test** is merged into **main** (production)



/ Multiverse_VSCode_GitHub_Practical_Collaboration

Q Type  to search



Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Label issues and pull requests for new contributors

Dismiss

Now, GitHub will help potential first-time contributors [discover issues](#) labeled with good first issue



feature/demo_feature_1 had recent pushes 3 minutes ago

Compare & pull request



feature/demo_feature_2 had recent pushes 1 minute ago

Compare & pull request

Showing 1 changed file with 5 additions and 0 deletions.

Split Unified

5 new_employee_data.csv

@@ -0,0 +1,5 @@

```
1 + EmployeeID,Name,Department,Salary
2 + 1001,Alice,Sales,32000
3 + 1002,Ben,Marketing,35000
4 + 1003,Chloe,IT,42000
5 + 1004,David,Finance,39000
```



No conflicts with base branch

Merging can be performed automatically.

Merge pull request



You can also merge this with the command line.

[View command line instructions.](#)

Still in progress? [Convert to draft](#)

▼ **GRAPH**



Auto



Merge pull request #1 from lauramart...



main



adding new employee data file to feature branch 1...



Update .gitignore Laura Martin



.gitignore

M



Instructional comments added to demo_script_1.py L...



Remove cached .env Laura Martin



Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons](#)

[t](#)



This branch has conflicts that must be resolved

Use the [web editor](#) or the command line to resolve conflicts before continuing.



demo_script_1.py

Resolve conflicts

Merge pull request



You can also merge this with the command line.
[View command line instructions.](#)



demo_script_1.py

1 conflict



Prev ^

Next v



Mark as resolved

```
1  import csv
2  import pandas as pd
3
4  with open('data.csv', 'r') as file:
5      reader = csv.reader(file)
6      for row in reader:
7          print(row)
8
9
10 <<<<< data/0_demo_script_2 (Current change)
11 df = pd.read_csv('data.csv')
12 print(df)
13 =====
14 # Intentional conflict to demonstrate GitHub collaboration. This line will be changed in the other branch.
15 >>>>> main (Incoming change)
16
17 # run python demo_script_1.py in the terminal to execute the
18 # script and see the output of the CSV file.
```



Intentional conflict merge: adding pandas dataf... #2

Resolving conflicts between `feature/demo_featu...` and `main` and committing changes → `feature/demo_featu...`

Commit merge

1 conflicting file

demo_script_1.py
demo_script_1.py



demo_script_1.py

✓ Resolved

```
1  import csv
2  import pandas as pd
3
4  with open('data.csv', 'r') as file:
5      reader = csv.reader(file)
6      for row in reader:
7          print(row)
8
9
10 df = pd.read_csv('data.csv')
11 print(df)
12
13 # run python demo_script_1.py in the terminal to execute the
14 # script and see the output of the CSV file.
```



Delete merged branch:

```
git branch -d feature/new-dashboard
```

Delete all deleted remote branches:

```
git fetch --prune
```

Remove local references to deleted branches:

```
git remote prune origin
```

Recommended Workflow

Main → Dev → Feature Branch → Pull Request into Dev → Dev/Test → Pull Request → Main

main = production-ready code

dev/test = integration/testing area

feature branch = safe workspace for one change

Recommended Workflow

Main → Dev → Feature Branch → Pull Request into Dev → Dev/Test → Pull Request → Main

Never:

- Work directly on main
- Leave branches open for months
- Create **huge** pull requests
- Ignore merge conflicts

Always:

- Pull before starting work
- Use descriptive commit messages
- Create feature branches
- Review pull requests
- Merge regularly
- Delete completed branches